

Software Patterns

Software Patterns - typical solutions to commonly occurring problems in software design.



Creational
(object creation)

Structural
(как собрать объекты
и классы в боковые
структуры)

Behavioral
(коммуникация
и закон отворота.)

Structural - фокусируется на организации классов и объектов. Помогают оптимизировать структуру иерархии и композициям, чтобы упростить взаимодействие объектов, уменьшить дублирование и создать новые структуры.

Behavioral - фокусируется на том, как объекты взаимодействуют с другими и как распределены обязанности. Их цель - упростить взаимодействие объектов и реализовать определенные алгоритмы и способы обмена данными, разграничить responsibilities.

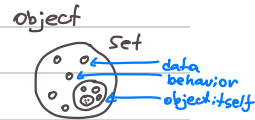
Design patterns address non-functional requirements

Basic Concepts

SUT service under test - test, whether response, the service behavior meets its expected requirements or specifications.

Encapsulation:

- Objects are a self-contained set of data & behavior
- Object can determine which part of its data & behavior is exposed to the outer world.
public, private, protected
- What an object exposes is called public interface.



Abstract classes define what needs to be shared among sub classes.

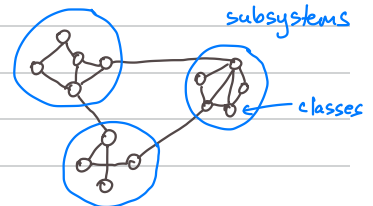
Abstraction allows to define an abstract structure which holds common states, behaviors or attributes.

Coupling ^{← *связанность*} measures the dependencies between subsystems.

Cohesion [↑] measures the dependencies among classes within a subsystem.

[↑]
"связанность"

← *классы и связанность*



Low coupling, high cohesion!

Если меньше одна подсистема, с
одинаковым числом не происходит!

Чтобы это добиться, нужно исп.



Law of Demeter

principle of
least knowledge

Class O:

M может использовать только:

M()

- Другие методы $\approx$ O
- Методы объектов, передаваемых в M как параметры
- Методы объектов, созданных внутри M
- Методы других объектов O

objectO.doSmith() - OK

objectO.getObjectP().doSomething() - НЕ OK

Ask, don't tell - просим у объекта что-то сделать:
turn.addStudent(student, Courses.PSE)

Polymorphism:

- Реализация абстракции и интерпретация разными способами
- Dynamic treatment of objects based on their type
- Перегрузка методов

Параметризованные типы

ArrayList<E> - generic type

↑
нотация для парамет. типа

Generic operations - операции над генериками

(Продолжение W01. Basic Concepts 2)

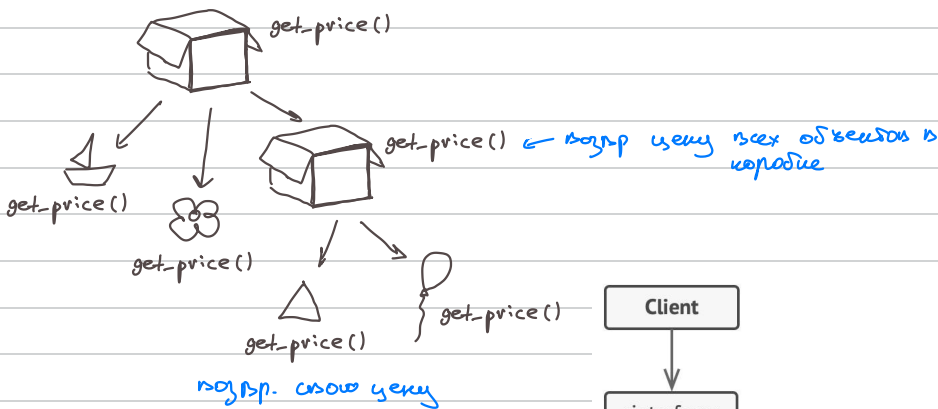
Composite Pattern

structural

Позволяет собирать объекты в древостоеподобные структуры и затем работать с этими структурами, как с отдельными объектами.

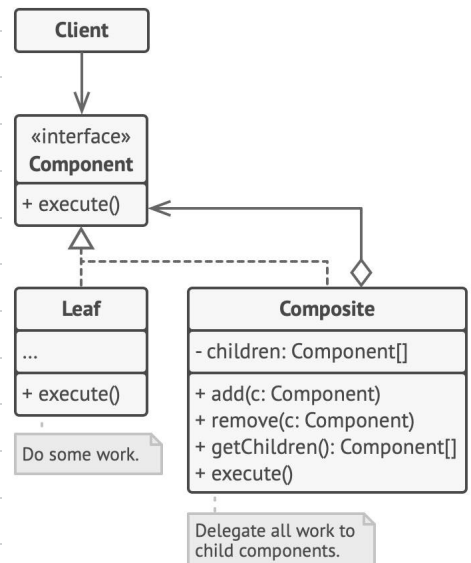
→ всего 2 типа объектов!

Каждый объект (нода или лист) имплементирует интерфейс:



Когда использовать:

- Нужно имплемент. tree-object structure (2 типа объектов: простые и составные)
- Нужно обрабатывать простые и составные объекты одинаково



Facade Pattern

structural

Предоставляет упрощенный интерфейс к библиотеке, принимающей или подобной огромной сложной коллекции классов.

Client



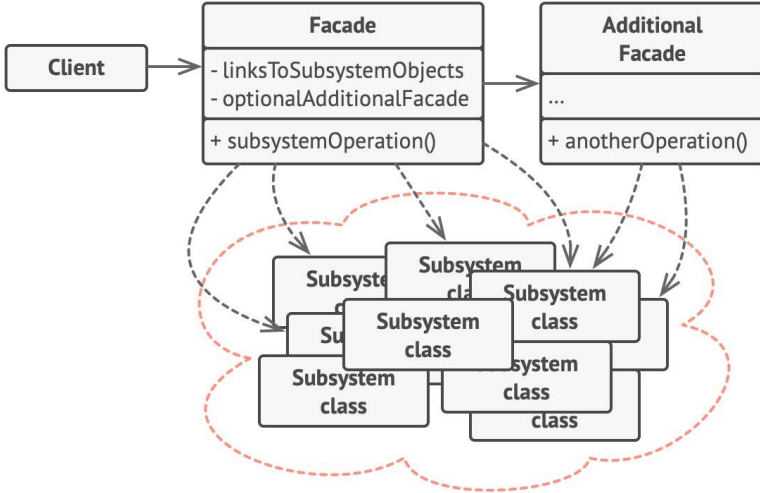
Billing

quotes

gry

compute

...



Bridge Pattern

structural

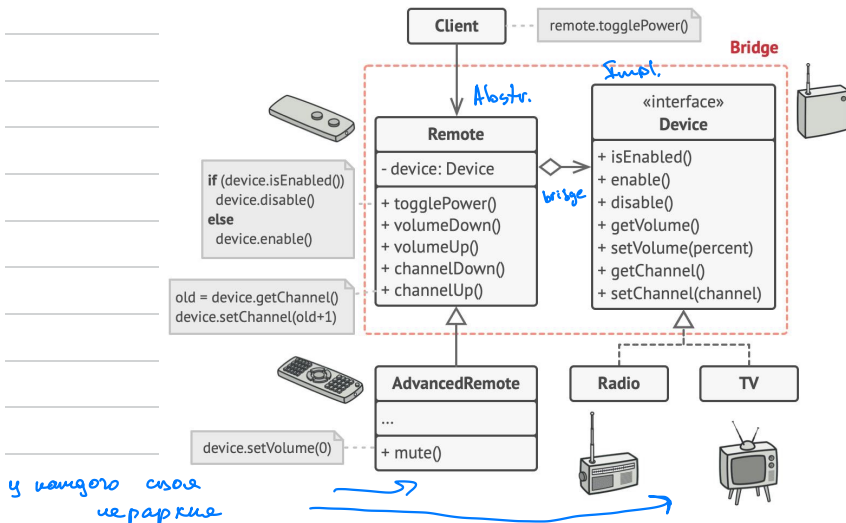
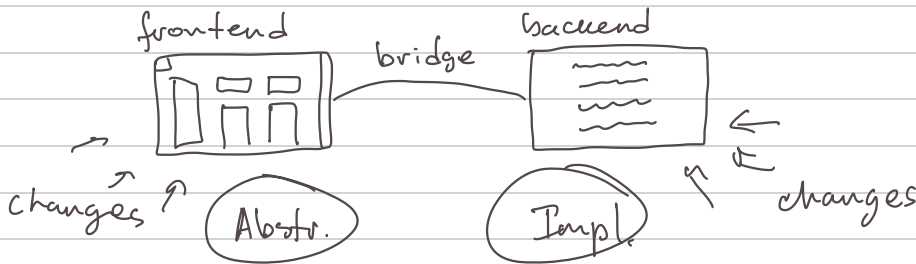
Позволяет поделить большой набор родственных классов на 2 отдельных иерархии - abstraction и implementation - которые можно менять независимо друг от друга

Abstraction (or interface) - high-level control layer for some entity.
Delegates the work to the **implementation** layer.

GUI

API

Не то же самое, что абстракция и интерфейс в проп.



Реализация

```
class Renderer:
```

```
    def render_circle(self, radius):  
        pass
```

```
class VectorRenderer(Renderer):
```

```
    def render_circle(self, radius):  
        print(f"Drawing a circle with radius {radius} in vector format")
```

```
class RasterRenderer(Renderer):
```

```
    def render_circle(self, radius):  
        print(f"Drawing a circle with radius {radius} in raster format")
```

Абстракция

```
class Shape:
```

```
    def __init__(self, renderer):  
        self.renderer = renderer
```

```
    def draw(self):  
        pass
```

```
class Circle(Shape):
```

```
    def __init__(self, renderer, radius):  
        super().__init__(renderer)  
        self.radius = radius
```

```
    def draw(self):  
        self.renderer.render_circle(self.radius)
```

Применение

```
vector_renderer = VectorRenderer()
```

```
circle = Circle(vector_renderer, 5)
```

```
circle.draw() # Вывод: "Drawing a circle with radius 5 in vector format"
```

.l.

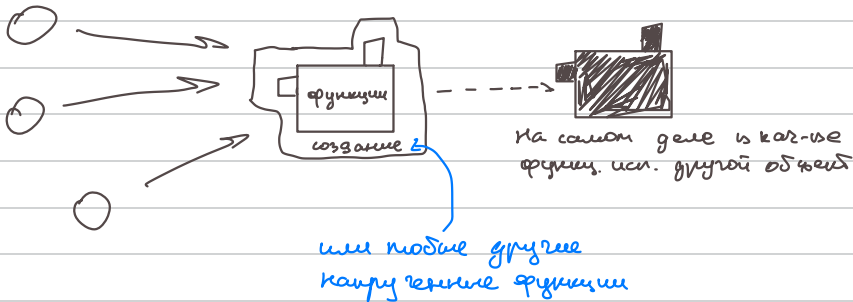
Когда использовать:

- Делим объект, у которого есть нек-е. пар. одной и той же функциональности
- Расширить класс в несколько независ. измерений (разные иерархии)
- Хотелось иметь имплементации в рантайме

Proxy Pattern

structural

Allows you to create a "placeholder" for another object to control it. This allows you also to perform sth either before or after the request gets through the original object.



Remote Proxy - управл. доступом к объекту на удаленном сервере.

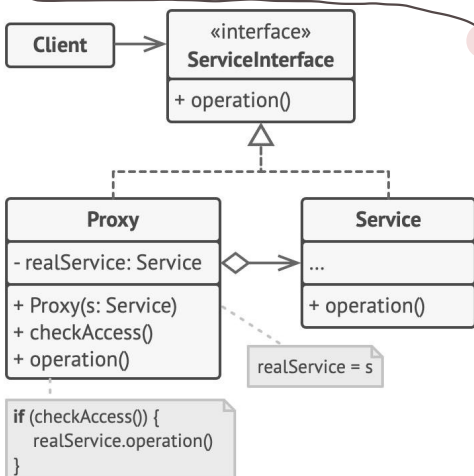
Virtual Proxy - отложенное создание (lazy initialization)

Protection Proxy - доступ на основании прав

Logging Proxy - копирует каждое обращение к объекту

Caching Proxy - кэширует повтор. запросы к объекту

Smart Reference - прокси считает "умные" ссылки на объект и предлагает помощь, если их не осталось.



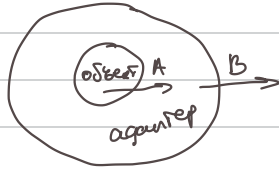
Proxy $\Leftrightarrow$ Object
взаимозаменяем.

т.к. один интерфейс,
и отн. от фасада

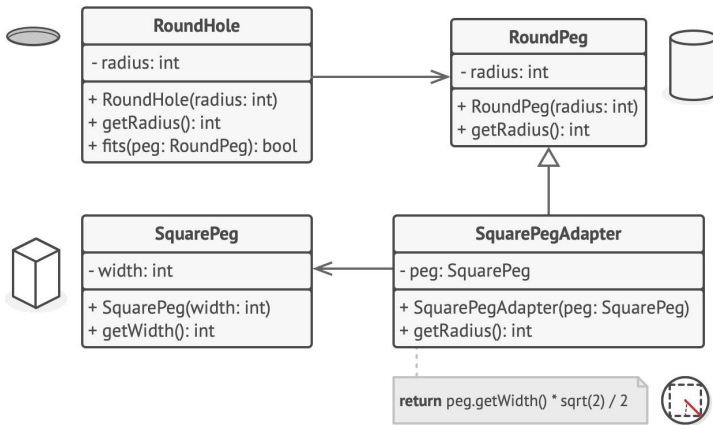
Adapter Pattern

structural

Позволяет объектам с разным интерфейсом взаимодействовать.

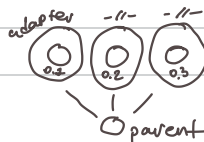


Иногда даже можно создать "two-way" adapter.



Когда использовать:

- Можно исп. сущ. класс, но его интерфейс не подходит
- Можно исп. несколько сущ. "подклассов", у которых нет common functionality, и ее нужно добавлять в общий родительск. класс



Strategy Pattern

behavioral

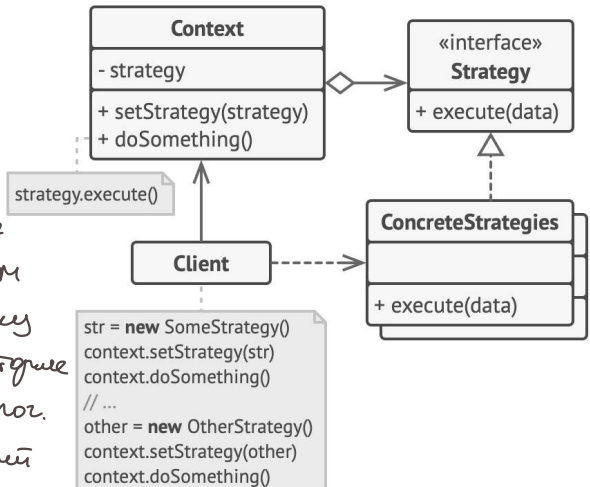
Позволяет задать семейство алгоритмов, поместить каждый из них в отдельный класс и сделать их экземпляры взаимозаменяемыми.



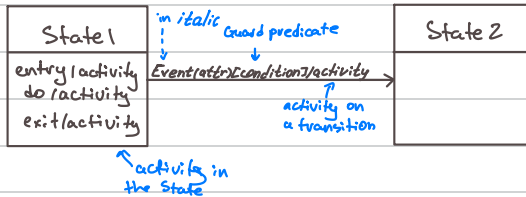
Клиент сам выбирает алгоритм и передает в калькулятор. Калькулятор его только применяет, но клиент не знает про алгоритм.

Когда использовать:

- Хотим иметь варианты алгоритма в runtime
- Есть много классов, которые реализуются лишь алгоритмом
- Нужно изобр. бизнес логику от деталей реализации, которые совсем не важны для бизнес лог.
- Есть огромный if, который переключает м/д вариантами одного алгоритма



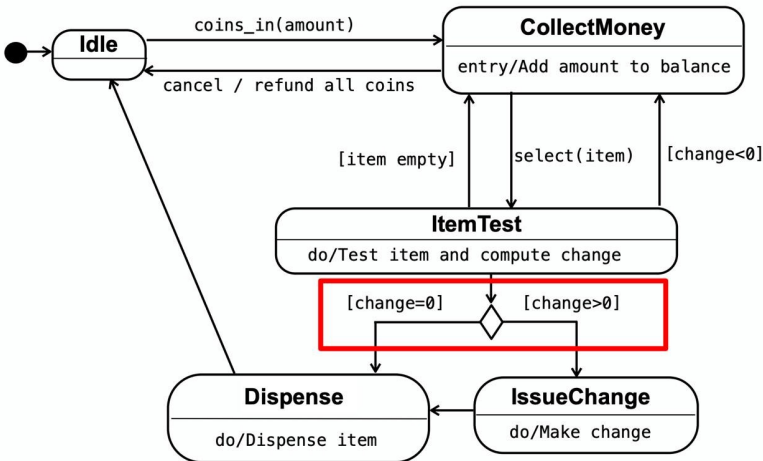
UML State Chart Diagram



Name Graphical Icon

Initial node	●
Final node	●
Fork node	→ →
Join node	→ →
Choice node	◇

Vending machine



To translate in Java code:

1. Write a **public method** for each event
2. Define a variable **state**
3. Write a **switch statement** with cases for each of the states in the UML state diagram
4. Write an **if statement** for each **guard predicate** on the transitions to check if it has become true
5. Update the **state** variable accordingly

State Pattern

behavioral

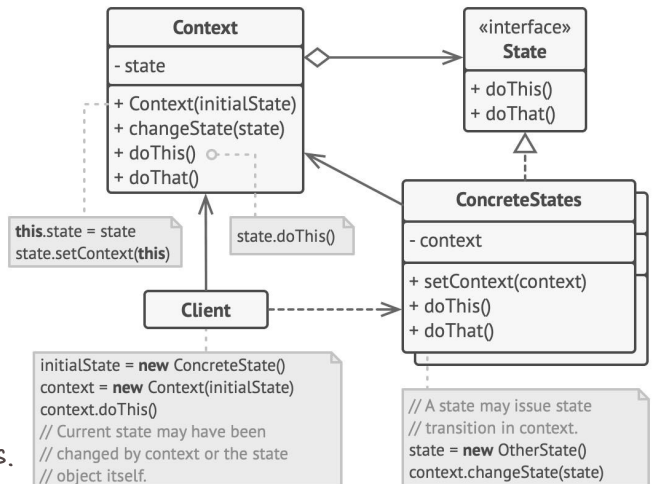
Позволяет объекту менять свое поведение, когда меняется его внутренний State, чтобы выиграло, как будто бы объект поменял класс.

Создаем по классу на каждый State объекта и записываем в них моды state-specific поведение. В основном объекте (Context object) сохраняем ссылки на текущ. объект состояния.

Похоже на Strategy Pattern, но там strategies нигде не знают друг о друге, а тут стейты могут инициать переходы друг в друга.

Когда использовать:

- Есть объект, который ведет себя по-разному в завис. от state, когда states большое и state-specific код трудно умен.
- Есть класс, в котором миллион if'ов, которые меняют поведение объекта в завис. от текущих значений class' fields.
- Много дубликатов кода внутри нескольких стейтов и переходов стейт-машины



В Context class создаем setter на стейт. Можно менять стейт из контекста, а можно внутри стейта создавать другой стейт и вызывать setter.

Template Method Pattern

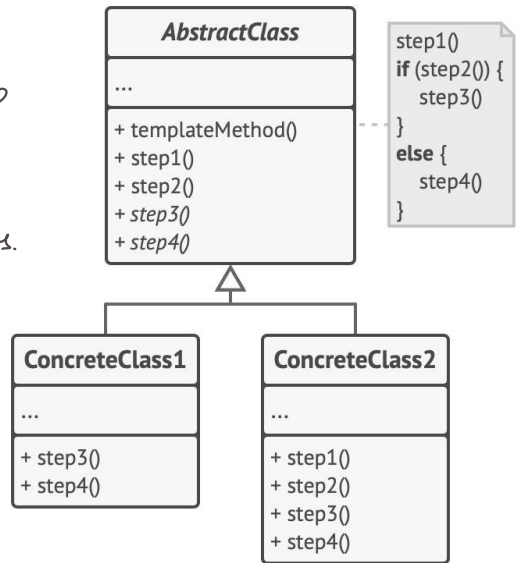
behavioral

Задаёт скелет алгоритма в родительском классе, но разрешает дочерним классам override конкретные шаги алгоритма без изменения его структуры.

Делим алгоритм на куски, записываем их в методы и вызываем их в единственном template method. У нас можно сделать абстрактными или задать им default implementation.

Когда использовать:

- Хотелось позволить изменить только некоторые шаги алгоритма, а не весь алгоритм или его структуру.
- Есть несколько классов, содержащих почти идентичные алгоритмы

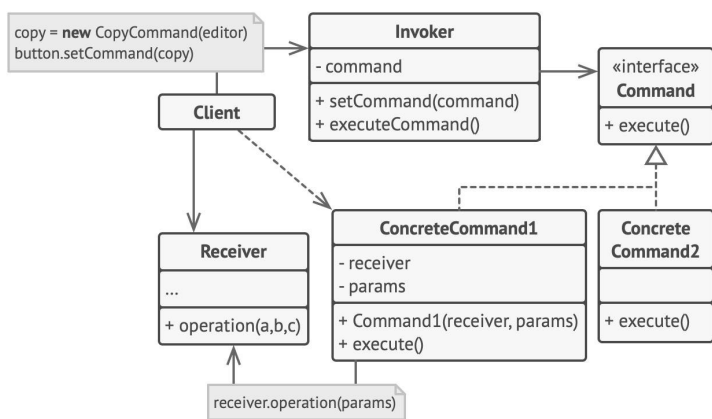


Command Pattern

behavioral

Преобразует request в объект, который содержит в себе всю информацию про request, что позволяет передавать request в как-либо аргумента функции, откладывая его или ставить в очередь.

Интерфейс определяет логику выполнения команды и сам объект команды, а потом просто вызывает ее метод execute внутри исполнителя команд.



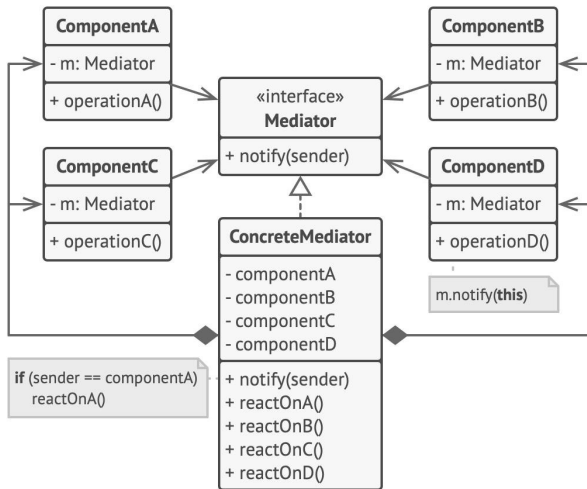
Когда использовать:

- Нужно передать команды (request'ы) в как-либо аргументов.
- Нужно ставить операции в очередь, отложить их выполнение или выполнить их удаление.
- Нужно уметь отменить действия (undo/redo) - просто делаем историю операций и состояний.

Mediator Pattern

behavioral

Позволяет снизить кол-во хаотичных зависимостей между объектами. Запрещает прямые коммуникации между объектами и обязывает их взаимодействовать только через Mediator Object.



Когда использовать:

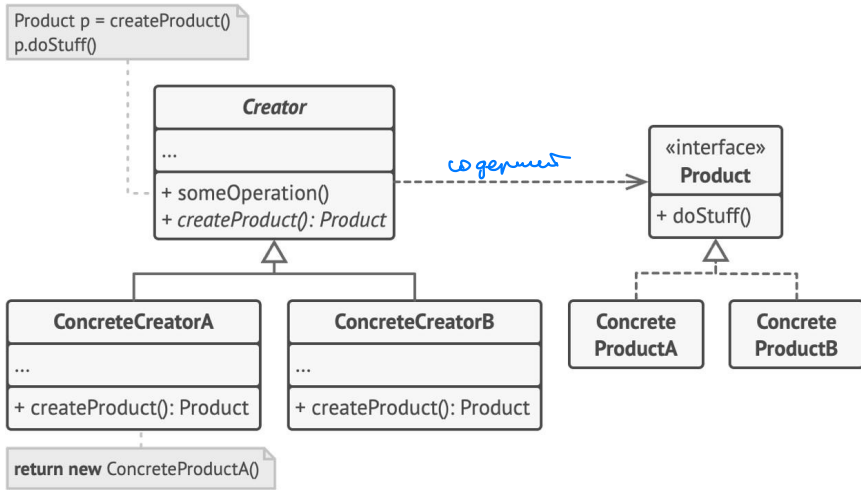
- Уже слишком много классов, потому что они очень сильно взаимодействуют друг с другом.
- Не получается переиспользовать компоненты — //

Нужно быть осторожным и не создавать Controller или God object

Factory Method Pattern

creational

Предоставляет интерфейс для создания объектов в родительском классе, но позволяет дочерям менять тип этих объектов.



Когда использовать:

- Заранее не известно, какие конкретно типы и зависимости объектов будут в коде
- Хотелось дать юзерам библиотеки/фреймворка возможность расширять/изменять ее внутренние компоненты.
- Хотелось сэкономить системные ресурсы, не пересоздавая объекты каждый раз, а используя существующие.

+ avoid tight coupling

Abstr. F. more coupl.

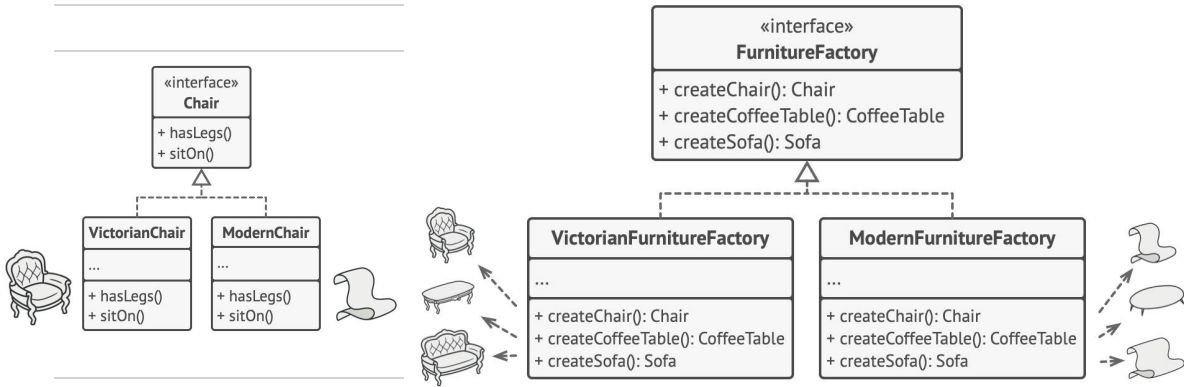
+ single respons.

+ introduce new traits

Abstract Factory Pattern

creational

Позволяет создавать семейства related объектов без того, чтобы указывать их конкретный класс.

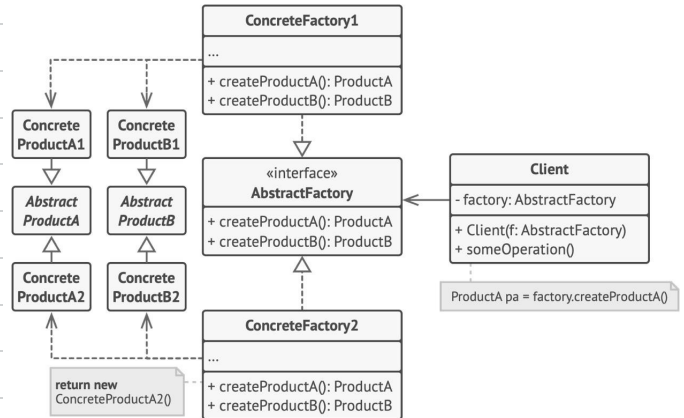


Когда использовать:

- Нужно работать с разнотипными семействами related объектов, но не хочется записывать от конкретного класса этих объектов (например,

когда они заранее не известны или в будущем там нужна flexibility.

- Есть класс с набором абстрактных методов, которые "размывают" его основную ответственность (т.е., например, когда класс deals with multiple product types)



Builder Pattern

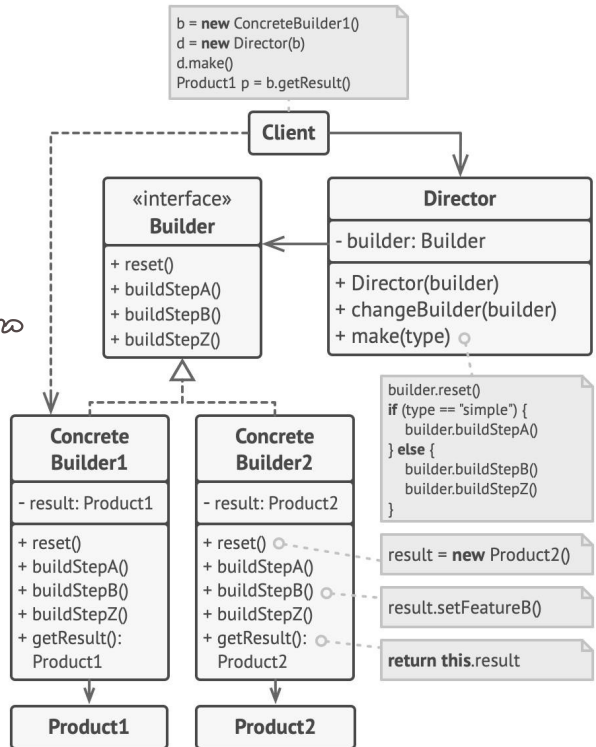
creational

Позволяет конструировать сложные объекты шаг за шагом и производить разные типы и representations объектов, используя один и тот же construction код.

По сути классом-бюджером является Директор, который определяет порядок шагов для бьюдера (какие шаги он выполняет).

Когда использовать:

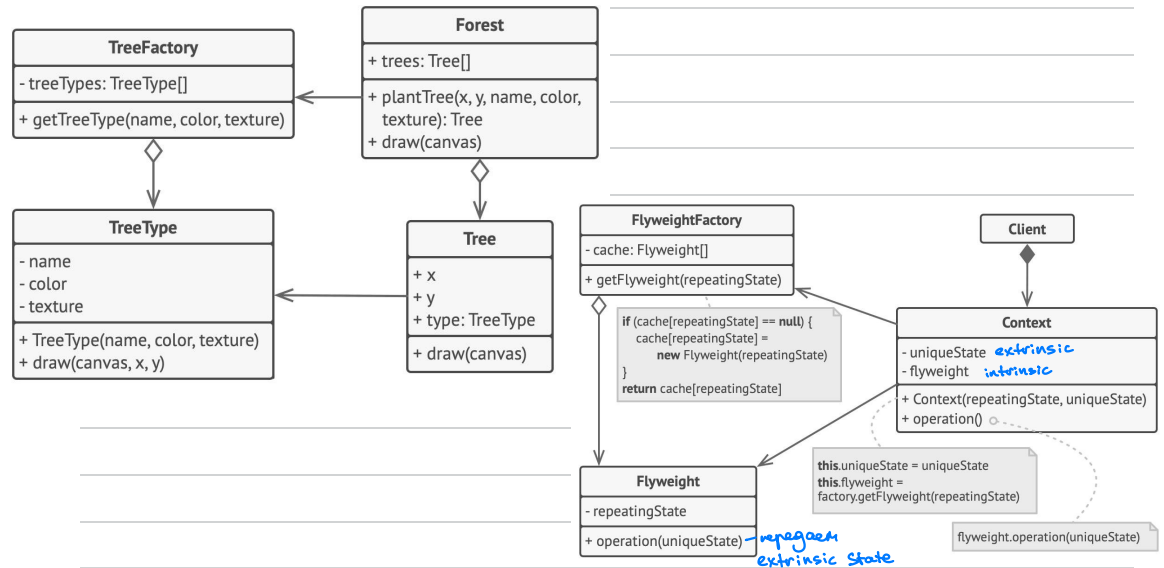
- Хотелся избавиться от большого количества параметров конструктора: $A(int a, int b, int c, int d, \dots)$
- Хотелся уметь создавать разные representations одного продукта (каменный дом и деревянный дом)
- Надо строить **КОМПОЗИТНЫЕ** деревья или другие сложные объекты



Flyweight Pattern (aka Cache Pattern)

structural

Позволяет хранить большое количество объектов в RAM, используя их общие ресурсы.



intrinsic - присущий, свойственный, внутренний

extrinsic - внешний, несвойственный

Когда использовать:

- Только когда программа использует огромное количество объектов и они не меняются в памяти.

Architectural Patterns

Buschmann и другие определяют:

Architecture style - в целом структурные фреймворки - макросы, ^{или микросервис} микросервисы.

Architectural pattern - структура приложения внутри приложения. стиль - MVC, Events...

Styles - независимые друг от друга.

Patterns - зависят от более мелких паттернов, более проблемно-ориентиров.

PSE определение:

Architecture style == architecture pattern - паттерн для subsystem
генерализованные

Software architecture - instance of an architectural pattern

Software architectures are instances of architectural styles, which are patterns for a subsystem decomposition based on specific design goals.

Хорошая архитектура:

- Результат конкретного применения набора принципов и техник ко всем фазам проекта
- Устойчива к неизбежным изменениям
- Позволяет повторно использовать "накопленные знания":

Применять архитектурные паттерны
теперь!

Components (subsystems) - obj, filters, layers, objects - computational units with specified interfaces

Connectors (communication) - method calls, pipes, shared data, event broadcasts, etc

MVC

Interactive systems

Model View Controller

Продукта: менуем ui, менуемца u data (entity objects)

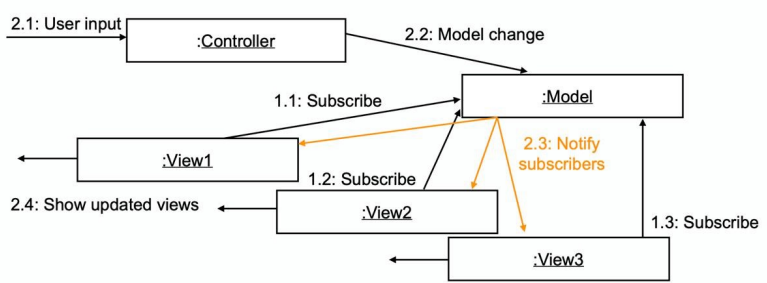
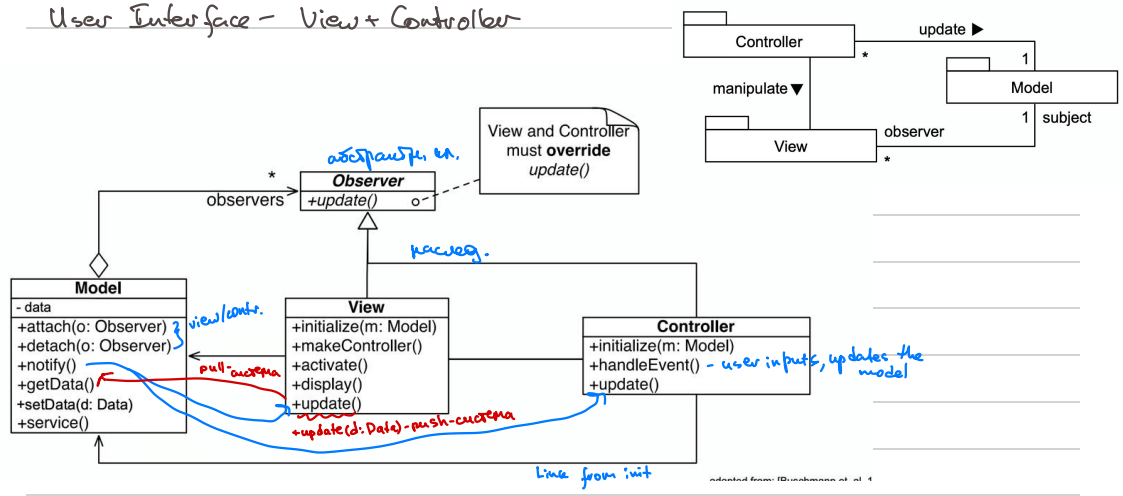
Резултат: генеруем entity objects (объекти на екрана) u boundary objects (ui)

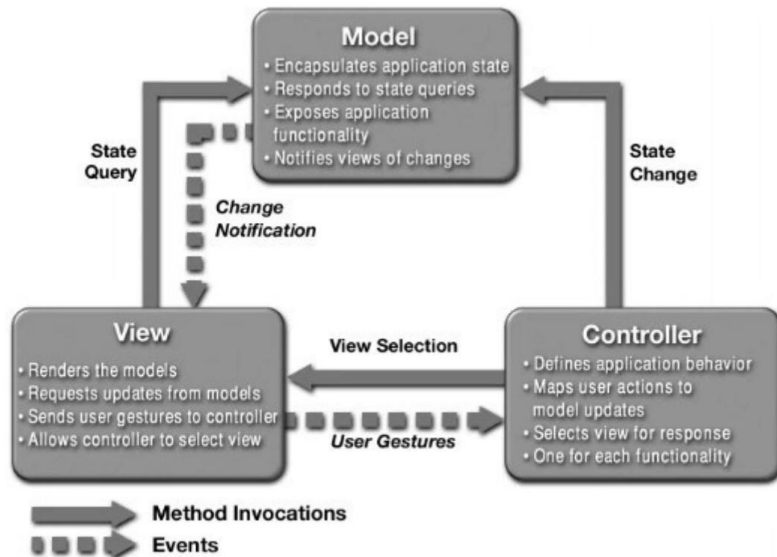
Model - domain (entity) objects

View - display information to the users (boundary objects)

Controller - interact with the user & update the model (which notifies the view)

User Interface - View + Controller





- **Benefits**

- Multiple synchronized views of the same model
- Pluggable views and controllers
- Exchangeability of look and feel
- Framework potential

- **Challenges**

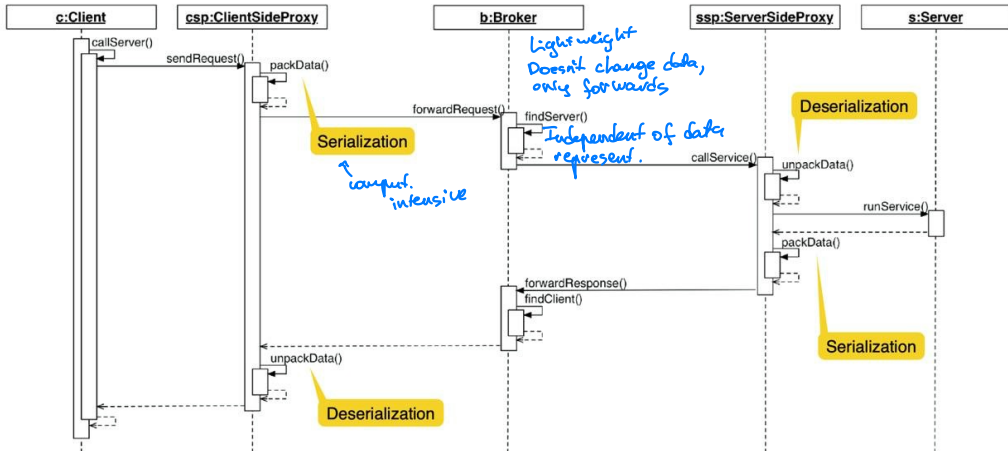
- Increased complexity
- Potential for excessive number of updates
- Close connection between view and controller
- Close coupling of views and controllers to model

Depends on the used version of MVC

Broker Pattern

Distributed Systems

Use of serialization and deserialization in the broker



Heterogeneous Architecture - components require different Software, hardware e.g. 4 processors or 4 processors need

Blackboard Pattern

From mind to structure

Универсальная для решения сложных задач с multiple source of truth.

Это Blackboard - модальная структура данных, кот. позволяет как shared workspace, где solutions are incrementally built. Хранит историю, достигнув решения и final result.

Могут быть моды не взаимодей., а только и только в Blackboard.

6 steps to realize a blackboard



1. **Define the problem** (make sure there is no algorithmic solution for the problem)
 - Identify the application domain and specify the requirements
 - Identify the actors (in particular the end user)
2. **Define the solution space**
 - Specify top level and intermediate solutions (for higher and lower abstraction levels)
 - Identify partial/complete solution candidates: a complete solution solves the problem, partial solutions solve parts of the problem
3. **Identify the knowledge sources (KS):** Identify their input and outputs
4. **Define the blackboard:** identify all the representations that are needed to allow the KSs to read from and contribute to the blackboard (not every KS needs to understand every representation!)
5. **Define the control:** implement the problem-solving strategy (Who can construct hypotheses? Who decides the credibility of hypotheses? Who decides what is an acceptable solution?)
6. **Implement the knowledge sources (KS)**
 - Split each KS into a condition part and an action part
 - Use different technologies: rule based systems, neural networks, in fact any computational intelligence method, but also conventional procedures/functions

Все авторитетные компоненты в Blackboard и все возможные конкретное решение

REST

Distributed Systems

Representational State Transfer - set of principals for network apps.

RESTful Web Service - переговоры, обмен & хранение репрезентаций ресурсов с помощью гипертекста.

Client - component that invokes a RESTful web service

Connector:

- Request/response
- Stateless protocol - не хранит инф. о предыдущих запросах

Mandatory RESTful API constraints:

- use of server-client model
- use of a layered system architecture
- stateless request-response cycle
- use of uniform interface

HTTP - application layer protocol used to communicate with web services

HTTP messages consist of:

URI, HTTP method, header and message body

Post, Get, Put, Delete
create, update

"Safe" - does not modify an object

URI - uniform Resource Identifier (URL is a form of URI)
↓
locator

RESTful APIs:

- Collections of endpoints
- Endpoint is identified by URI path + HTTP method
- E/p handles HTTP messages to manipulate REST resources